Mathematical Background for `AccumulatR`

Niek Stevenson

Abstract

`AccumulatR` is built around one simple idea: each latent process has a random finishing time. Responses are defined by rules about which processes must finish, which must not finish, and which finish first. This note gives a compact mathematical description of that framework. The aim is not maximal generality. The aim is a notation that is simple enough to read from start to finish and precise enough to match the package.

1 Overview

The framework has four layers.

1. A *leaf* is a single latent finishing-time process.
2. A *pool* combines several sources into one new source.
3. An *outcome rule* says when a modeled response counts as having occurred.
4. An *observation layer* turns latent outcomes into the data we actually record, for example through mixtures, remapped labels, or ranked responses.

Everything is built from finishing times. Once an object has a finishing time, it can compete with other objects in exactly the same way.

2 Basic Notation

For any process or event A, write T_A for its finishing time. We then use

$$F_A(t) = \Pr(T_A \leq t), \qquad f_A(t) = \frac{d}{dt}F_A(t), \qquad S_A(t) = 1 - F_A(t).$$

So $F_A(t)$ is the probability that A has finished by time t, $f_A(t)$ is the density that A finishes at time t, and $S_A(t)$ is the probability that A is still unfinished at time t.

The same notation will be used for leaves, pools, and outcome rules. That is the main reason the framework stays manageable: every object is represented in the same way once its finishing-time distribution is known.

In this note, a *source* means any object with its own finishing time. The two basic source types are leaves and pools.

3 Leaves

A leaf is the basic latent process in the model. If a is a leaf, then T_a is its finishing time and F_a, f_a, and S_a are its CDF, density, and survival function.

At this level, the framework only needs a valid density–CDF pair. The package currently supplies the families

`lognormal`, `gamma`, `exgauss`, `LBA`, `RDM`.

Nothing in the later sections depends on a particular family. Later constructions only need the functions F_a, f_a, and S_a.

Absolute onset. A leaf may also have an absolute onset delay $d_a \geq 0$. If $T_a^\star$ is the baseline finishing time after the process has started, then

$$T_a = d_a + T_a^\star.$$

Equivalently,

$$F_a(t) = F_a^\star(t - d_a), \qquad f_a(t) = f_a^\star(t - d_a),$$

with both terms equal to 0 for $t \leq d_a$. So an absolute onset simply shifts the finishing-time distribution to the right.

4 Pools

A pool turns several sources into one new source. Suppose pool P has members with finishing times $T_1, \ldots, T_n$, and suppose the pool finishes when the kth member finishes. Then T_P is the kth order statistic of $T_1, \ldots, T_n$.

The defining identity is

$$F_P(t) = \Pr(T_P \leq t) = \Pr(\text{at least } k \text{ of } T_1, \ldots, T_n \text{ are } \leq t). \tag{1}$$

That is the cleanest way to think about a pool: by time t, the pool has finished exactly when enough of its members have finished.

If the pool members are exchangeable, with common CDF F and density f, then

$$F_P(t) = \sum_{m=k}^{n} \binom{n}{m} F(t)^m S(t)^{n-m}, \tag{2}$$

and

$$f_P(t) = \frac{n!}{(k-1)!(n-k)!}\, f(t)\, F(t)^{k-1} S(t)^{n-k}. \tag{3}$$

These are the usual formulas for the kth finisher.

The package also allows pools whose members have different distributions. The notation is then longer, but the idea is the same: a pool is simply another source with its own F_P, f_P, and S_P.

Three special cases matter often:

- $k = 1$: the pool is a first finisher.
- $k = n$: the pool is a last finisher.
- $n = 1$: the pool is just an alias for one source.

Pools can also be nested. Once a source has its own F, f, and S, it can be used as a member of a larger pool.

5 Outcome Rules

An outcome rule says when a modeled response becomes available. If E is an outcome rule, then T_E is the first time that rule becomes true.

5.1 Direct Outcomes

The simplest outcome rule is a direct readout from one source. In that case, the outcome simply inherits the finishing-time distribution of that source. Nothing new is added mathematically.

5.2 `first_of()`

If

$$E = \texttt{first_of}(E_1, \ldots, E_m),$$

then the rule becomes true when the first child becomes true. Therefore

$$T_E = \min(T_{E_1}, \ldots, T_{E_m}),$$

and

$$F_E(t) = 1 - \prod_{r=1}^{m} S_{E_r}(t), \qquad f_E(t) = \sum_{r=1}^{m} f_{E_r}(t) \prod_{q \neq r} S_{E_q}(t). \tag{4}$$

The density has the usual interpretation: one child finishes at time t, and all the others are still unfinished at time t.

5.3 `all_of()`

If

$$E = \texttt{all_of}(E_1, \ldots, E_m),$$

then the rule becomes true when the last required child becomes true. Therefore

$$T_E = \max(T_{E_1}, \ldots, T_{E_m}),$$

and

$$F_E(t) = \prod_{r=1}^{m} F_{E_r}(t), \qquad f_E(t) = \sum_{r=1}^{m} f_{E_r}(t) \prod_{q \neq r} F_{E_q}(t). \tag{5}$$

Here one child finishes at time t, and all the others have already finished by time t.

5.4 `none_of()`

`none_of()` is an absence condition. If a rule requires `none_of(B)`, then the relevant term is simply the survival function of B:

$$\Pr(B \text{ has not finished by time } t) = S_B(t).$$

For example, if

$$E = \texttt{all_of}(A, \texttt{none_of}(B)),$$

then by time t the rule holds with probability

$$F_E(t) = F_A(t) S_B(t).$$

So `none_of()` does not introduce a new race winner. It contributes a survival factor.

5.5 `inhibit()`

The blocking rule is written as

$$G = \texttt{inhibit}(A, B).$$

Its meaning is simple: count A only if B has not finished first. The corresponding density is

$$f_G(t) = f_A(t) S_B(t), \qquad F_G(t) = \int_0^t f_G(u)\, du. \tag{6}$$

So A finishes at time t, while B is still unfinished at time t.

6 Trial Likelihood

Suppose the model defines outcomes $R_1, \ldots, R_J$. If response r is observed at time t, then the trial contribution is

$$\ell_r(t) = \frac{d}{dt} \Pr\big(T_{R_r} \le t,\ T_{R_j} > t \text{ for all } j \ne r\big)\,. \tag{7}$$

This is the general "response r wins at time t" formula.

6.1 Disjoint outcomes

In the standard case, the outcome rules are disjoint. Then Equation (**??**) reduces to

$$\ell_r(t) = f_{R_r}(t) \prod_{j \ne r} S_{R_j}(t). \tag{8}$$

This is the familiar race-model likelihood: response r finishes at time t, and every competing response is still unfinished at time t.

6.2 Overlapping outcomes

Some outcome definitions can overlap, so that the same latent path satisfies more than one rule. In that case the simple product formula is not enough.

If this section is not relevant for the model at hand, it can be skipped on a first reading. The point is just that overlap is handled by an inclusion–exclusion correction.

Let $\mathcal{C}_r = \{1, \ldots, J\} \setminus \{r\}$ be the competitor set for outcome r. Then

$$\ell_r(t) = \sum_{A \subseteq \mathcal{C}_r} (-1)^{|A|} f_{r \wedge A}(t), \tag{9}$$

where $f_{r\wedge A}(t)$ is the density that outcome r and every outcome in A are all true at time t.

The interpretation is straightforward even if the notation is not. The formula adds and subtracts overlap terms so that shared latent mass is counted once.

No-response mass. Some models leave positive probability on "no observed response", for example because a trigger fails. Before any relabelling or guessing, that probability is

$$p_{\text{none}} = 1 - \sum_{r=1}^{J} \int_0^\infty \ell_r(t)\, dt. \tag{10}$$

So the no-response mass is whatever probability remains after summing the finite response probabilities.

7 Timing Modifiers

7.1 Chained onsets

A leaf may start only after another source has finished. If leaf a starts after source U, with an additional lag $\delta \geq 0$, then

$$T_a = T_U + \delta + T_a^\star,$$

where $T_a^\star$ is the leaf's own post-start finishing time.

So the total waiting time is: wait for U, then wait for the lag, then wait for a itself. This gives the convolution

$$f_a(t) = \int_0^{t-\delta} f_U(s)\, f_a^\star(t-\delta-s)\, ds, \tag{11}$$

and similarly

$$F_a(t) = \int_0^\infty F_a^\star(t-\delta-s)\, f_U(s)\, ds. \tag{12}$$

7.2 Triggers

Triggers control whether a source starts at all.

Independent trigger. If leaf a has start probability $1 - q_a$, then the finite-time distribution is simply scaled by that probability:

$$F_a(t) = (1 - q_a)F_a^{\dagger}(t), \qquad f_a(t) = (1 - q_a)f_a^{\dagger}(t), \tag{13}$$

where $F_a^{\dagger}$ and $f_a^{\dagger}$ are the channels the leaf would have had without trigger failure.

Shared trigger. If several leaves share one trigger with failure probability q_g, then they either all start together or all fail together. The trial likelihood is then a two-part mixture:

$$L = (1 - q_g)L^{(\mathrm{on})} + q_g L^{(\mathrm{off})}. \tag{14}$$

This is the only point at which the framework introduces joint start dependence across several leaves.

8 Observation Layer

8.1 Mixture components

A component c defines one variant of the model, with weight π_c. If the component is observed on a trial, then the trial contribution is simply

$$L = L_c.$$

If the component is latent, then

$$L = \sum_c \pi_c L_c, \qquad \sum_c \pi_c = 1. \tag{15}$$

So observed components condition on one variant, whereas latent components average over variants.

8.2 Outcome remapping and guessing

The outcome generated by the model need not be the same as the label that is observed. Let M_{yr} be the probability that latent outcome r is recorded as observed label y. Then

$$\ell_y(t) = \sum_r M_{yr}\, \ell_r(t). \tag{16}$$

So the observed density is a weighted sum of the latent outcome densities.

If the observed label is kept but the response time is discarded, then the contribution becomes the integrated mass

$$p_y = \int_0^\infty \ell_y(t)\, dt. \tag{17}$$

8.3 Ranked responses

The package can also use data in which the first K responses are observed on each trial:

$$(r_1, t_1), \ldots, (r_K, t_K), \qquad 0 < t_1 < \cdots < t_K.$$

This part of the framework is narrower than the single-response case. The current ranked implementation is for direct, disjoint outcomes.

In that setting, the ranked likelihood is

$$L_{\text{rank}} = \left[\prod_{m=1}^{K} f_{r_m}(t_m)\right] \left[\prod_{j \notin \{r_1, \ldots, r_K\}} S_j(t_K)\right]. \tag{18}$$

Because the times are ordered, the sequence $t_1 < \cdots < t_K$ already encodes which response came first, second, and so on.

9 Scope in the Package

The mathematical framework above is the background needed to read the models currently supported by `AccumulatR`. In particular, the package includes

- leaf-level distributions,
- threshold pools, including nested pools,
- logical outcome rules such as `first_of()`, `all_of()`, `none_of()`, and `inhibit()`,
- absolute onsets and chained onsets,
- independent and shared triggers,
- fixed and latent mixtures,

- outcome remapping and guessing,
- ranked readout within the restrictions stated above.

The common theme is always the same: define finishing times for the latent pieces, build larger objects from them, and then write the likelihood in terms of which outcome becomes true first.